NAME : SHAH NAWAZ HUSSAIN
ROLL NO: 244CA049

LAB – 6 ASSGINMENTS

Q1.A 5-meter ladder is initially standing upright against a wall. Its bottom end is at (0,0), and its top end is at (0,5).

a. Now, the ladder undergoes the following transformations:. It is tilted 30◦ counterclockwise around its bottom end (0,0).

b. Then, the entire ladder is moved (translated) so that its bottom end is now at (3,2).

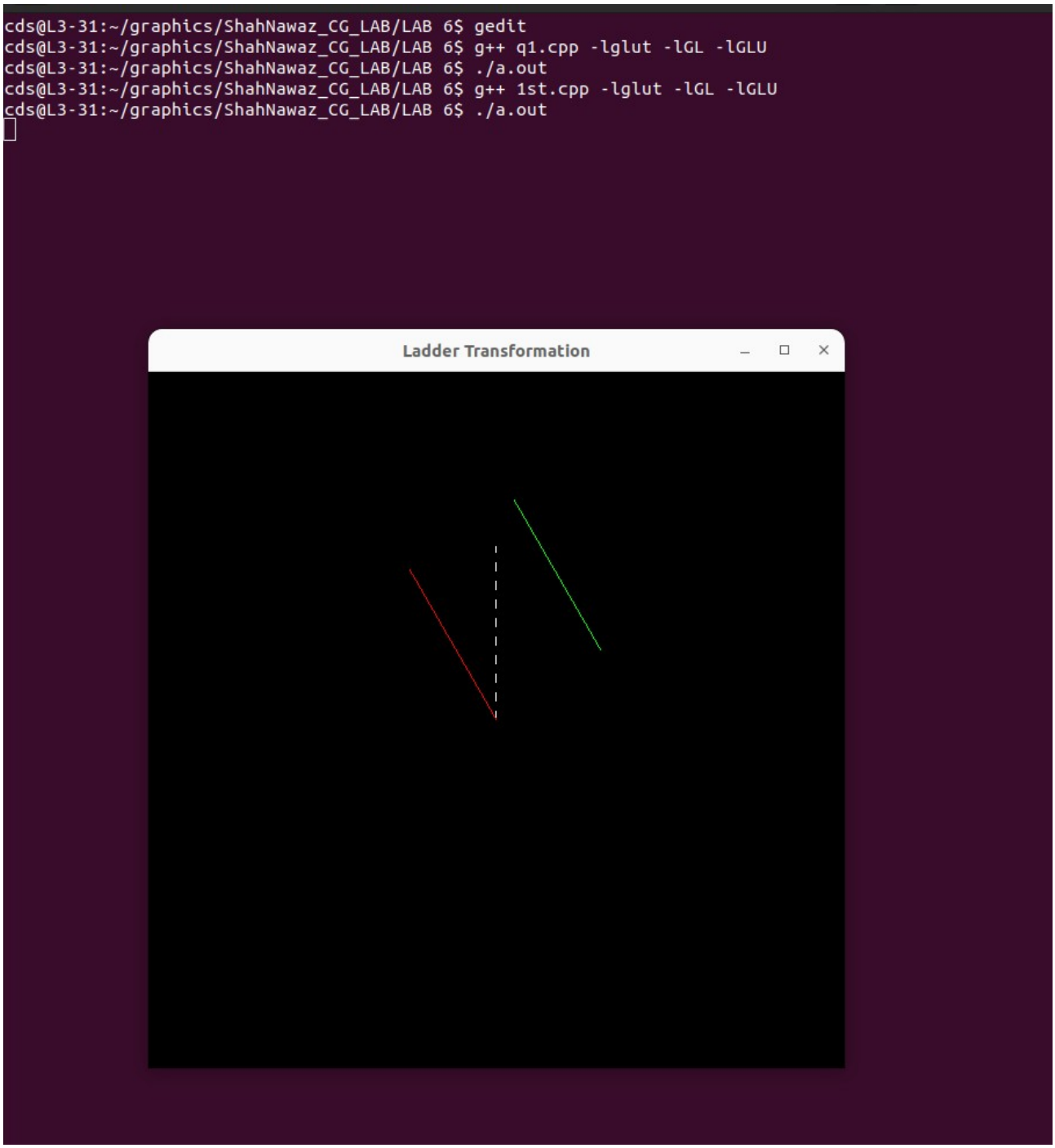
Apply necessary transformations and obtain the desired final figure.

Ans.1 CODE:

```
#include <GL/glut.h>
#include <math.h>

float x1 = 0, y_1 = 0;
float x2 = 0, y_2 = 5;

float rx2, ry2;
float tx1, ty1, tx2, ty2;

void compute() {
    float angle = 30 * M_PI / 180.0;

    rx2 = x2 * cos(angle) - y_2 * sin(angle);
    ry2 = x2 * sin(angle) + y_2 * cos(angle);


    tx1 = x1 + 3;
    ty1 = y_1 + 2;

    tx2 = rx2 + 3;
    ty2 = ry2 + 2;
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1,1,1);
    glEnable(GL_LINE_STIPPLE);
    glLineStipple(1, 0x00FF);

    glBegin(GL_LINES);
        glVertex2f(x1, y_1);
        glVertex2f(x2, y_2);
```

```
    glEnd();

    glDisable(GL_LINE_STIPPLE);

    glColor3f(1,0,0);
    glBegin(GL_LINES);
        glVertex2f(x1, y_1);
        glVertex2f(rx2, ry2);
    glEnd();

    glColor3f(0,1,0);
    glBegin(GL_LINES);
        glVertex2f(tx1, ty1);
        glVertex2f(tx2, ty2);
    glEnd();

    glFlush();
}

void init() {
    glClearColor(0,0,0,1);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10,10,-10,10);

    compute();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    glutInitWindowSize(600,600);
    glutCreateWindow("Ladder Transformation");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
}
```

OUTPUT:


cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ gedit
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ q1.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 1st.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
Ladder Transformation

Q2.Create and rotate a triangle about the origin and a fixed point.

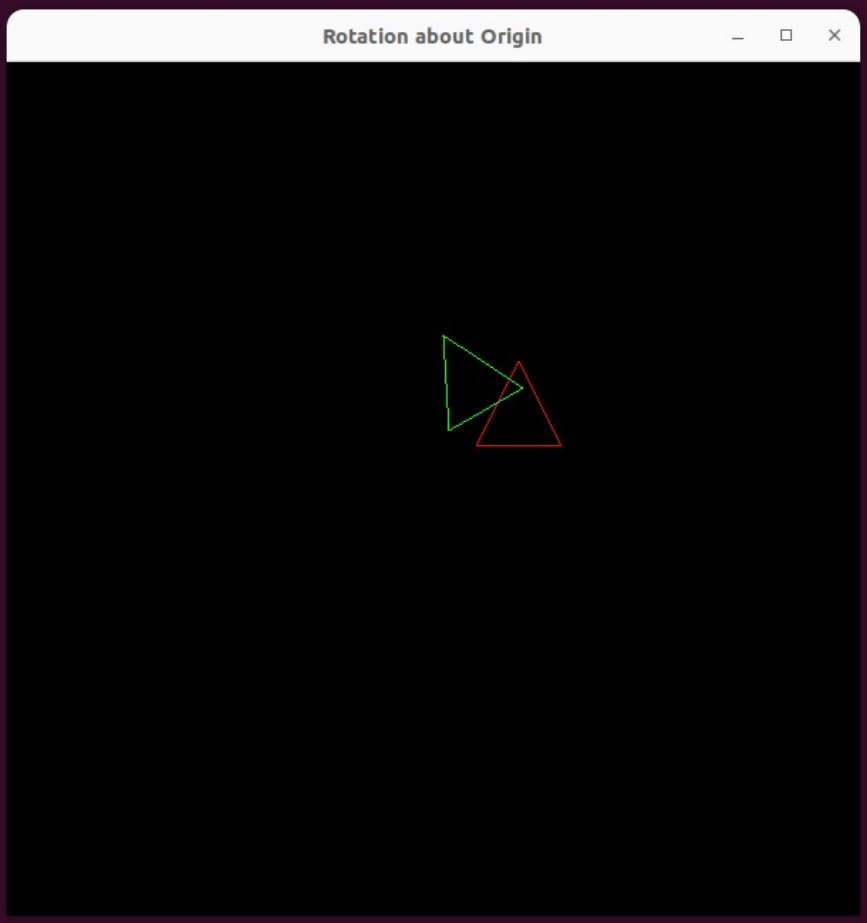
Ans.2
CODE:

```
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>

float x1 = 1, y_1 = 1;
float x2 = 3, y2 = 1;
float x3 = 2, y3 = 3;

float rx1, ry1, rx2, ry2, rx3, ry3;

float angle;

void compute() {
   float rad = angle * M_PI / 180.0;

   rx1 = x1 * cos(rad) - y_1 * sin(rad);
   ry1 = x1 * sin(rad) + y_1 * cos(rad);

   rx2 = x2 * cos(rad) - y2 * sin(rad);
   ry2 = x2 * sin(rad) + y2 * cos(rad);

   rx3 = x3 * cos(rad) - y3 * sin(rad);
   ry3 = x3 * sin(rad) + y3 * cos(rad);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glColor3f(1,0,0);
   glBegin(GL_LINE_LOOP);
      glVertex2f(x1, y_1);
      glVertex2f(x2, y2);
      glVertex2f(x3, y3);
   glEnd();

   glColor3f(0,1,0);
   glBegin(GL_LINE_LOOP);
      glVertex2f(rx1, ry1);
      glVertex2f(rx2, ry2);
      glVertex2f(rx3, ry3);
   glEnd();

   glFlush();
}

void init() {
```

```
    glClearColor(0,0,0,1);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10,10,-10,10);

    compute();
}

int main(int argc, char** argv) {

    printf("Enter rotation angle: ");
    scanf("%f", &angle);

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    glutInitWindowSize(600,600);
    glutCreateWindow("Rotation about Origin");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
}
```

OUTPUT:

```
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ gedit
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ q1.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 1st.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 2nd.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
Enter rotation angle: 30
```



Q3. Reflect a line first about the x-axis and then about the y-axis.

Ans3.

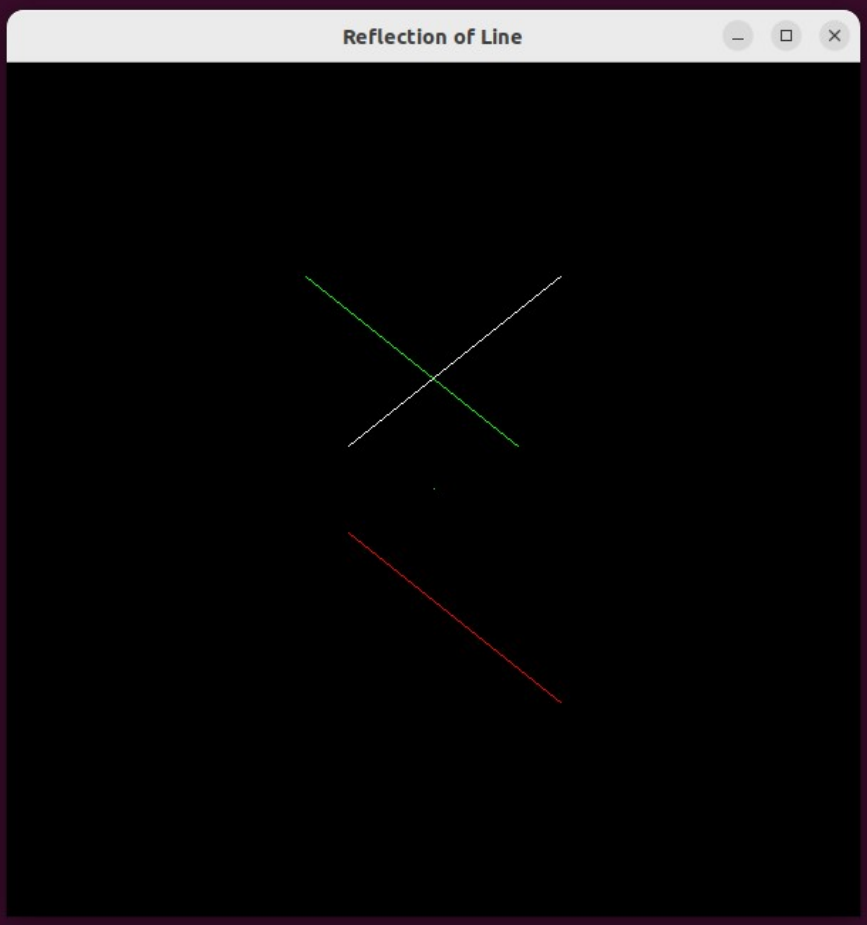
CODE:

```
#include <GL/glut.h>

float x1 = -2, y1_ = 1;
float x2 = 3, y2 = 5;

float rx1, ry1, rx2, ry2;

float fx1, fy1, fx2, fy2;

void compute() {

    rx1 = x1;
    ry1 = -y1_;

    rx2 = x2;
    ry2 = -y2;

    fx1 = -x1;
    fy1 = y1_;

    fx2 = -x2;
    fy2 = y2;
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1,1,1);
    glBegin(GL_LINES);
        glVertex2f(x1, y1_);
        glVertex2f(x2, y2);
    glEnd();

    glColor3f(1,0,0);
    glBegin(GL_LINES);
        glVertex2f(rx1, ry1);
        glVertex2f(rx2, ry2);
    glEnd();

    glColor3f(0,1,0);
    glBegin(GL_LINES);
        glVertex2f(fx1, fy1);
        glVertex2f(fx2, fy2);
    glEnd();
     glBegin(GL_POINTS);
        glVertex2f(0, 0);
```

```
    glEnd();

    glFlush();
}

void init() {
    glClearColor(0,0,0,1);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10,10,-10,10);

    compute();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    glutInitWindowSize(600,600);
    glutCreateWindow("Reflection of Line");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
}
```

OUTPUT:

```
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ gedit
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ q1.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 1st.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 2nd.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
Enter rotation angle: 30
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 3rd.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ g++ 3rd.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/ShahNawaz_CG_LAB/LAB 6$ ./a.out
```



Q4. You are given the following geometric shapes in a local coordinate system: a.

a. A rectangle (representing the house) with bottom-left corner at (0,0) and top-right corner at (6,9).

b. A triangle (representing the roof) with vertices at: A=(0,0), B=(2,0), C=(1,1) i. You need to:

c. Translate the rectangle so that its left edge is positioned at x=5 on the x-axis.

d. Place the triangle on top of the rectangle, ensuring that:

1. The base of the triangle aligns with the top edge of the rectangle.

ii. The triangle remains centered over the rectangle.

' Apply the transformations and show the final figure

Ans.4

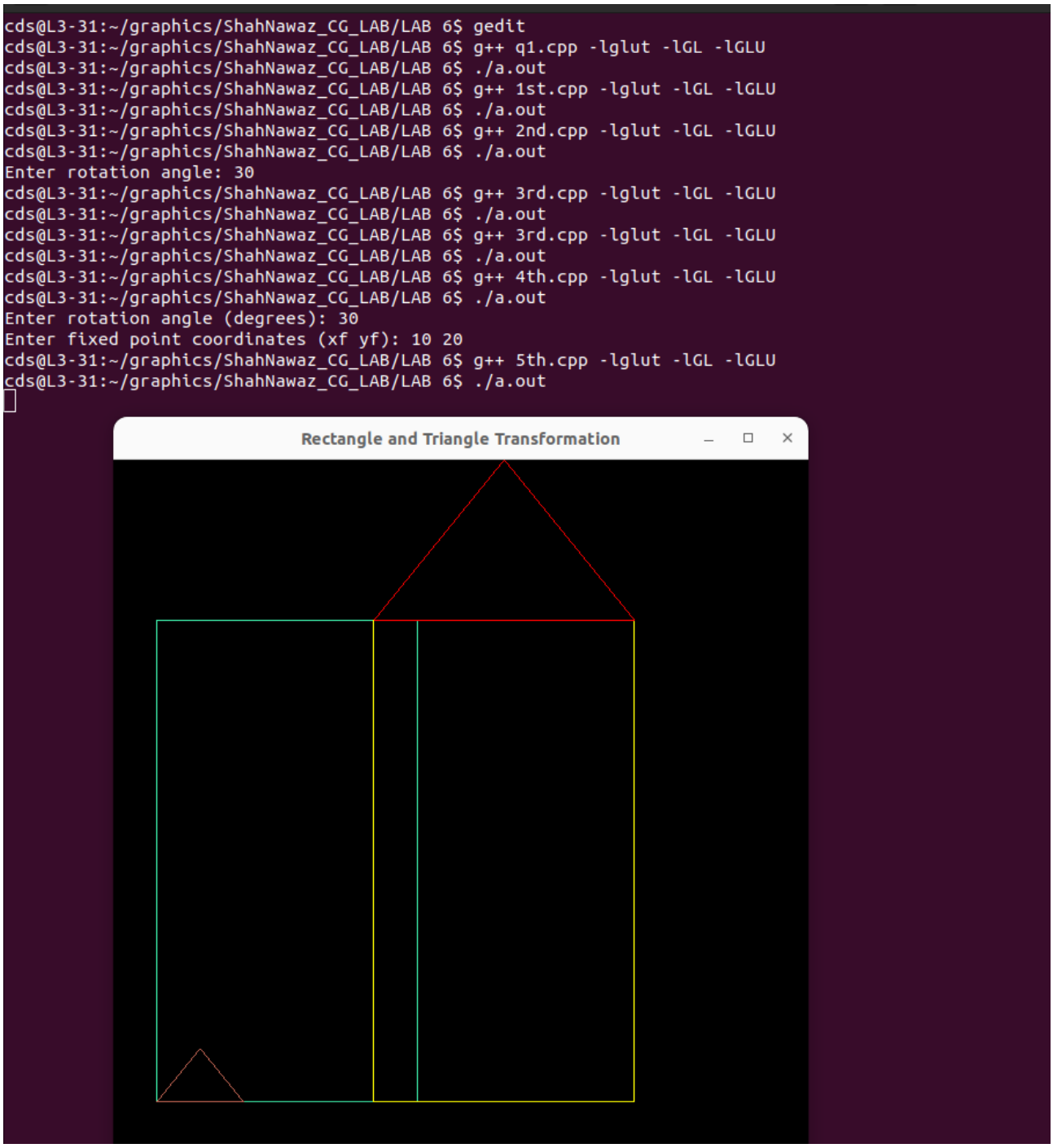
CODE:

```
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>

float x1 = 1, y1_ = 1;
float x2 = 3, y2_ = 1;
float x3 = 2, y3_ = 3;

float rx1, ry1, rx2, ry2, rx3, ry3;

float xf, yf;

float angle;

void computeRotation() {
    float rad = angle * M_PI / 180.0;

    float tx = x1 - xf;
    float ty = y1_- yf;
    rx1 = tx * cos(rad) - ty * sin(rad) + xf;
    ry1 = tx * sin(rad) + ty * cos(rad) + yf;

    tx = x2 - xf;
    ty = y2_ - yf;
    rx2 = tx * cos(rad) - ty * sin(rad) + xf;
    ry2 = tx * sin(rad) + ty * cos(rad) + yf;

    tx = x3 - xf;
    ty = y3_ - yf;
    rx3 = tx * cos(rad) - ty * sin(rad) + xf;
```

```
    ry3 = tx * sin(rad) + ty * cos(rad) + yf;
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1,1,1);
    glBegin(GL_LINE_LOOP);
        glVertex2f(x1, y1_);
        glVertex2f(x2, y2_);
        glVertex2f(x3, y3_);
    glEnd();

    glColor3f(0,1,0);
    glBegin(GL_LINE_LOOP);
        glVertex2f(rx1, ry1);
        glVertex2f(rx2, ry2);
        glVertex2f(rx3, ry3);
    glEnd();

    glPointSize(6);
    glColor3f(1,0,0);
    glBegin(GL_POINTS);
        glVertex2f(xf, yf);
    glEnd();

    glFlush();
}

void init() {
    glClearColor(0,0,0,1);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10,10,-10,10);

    computeRotation();
}

int main(int argc, char** argv) {
    printf("Enter rotation angle (degrees): ");
    scanf("%f", &angle);

    printf("Enter fixed point coordinates (xf yf): ");
    scanf("%f %f", &xf, &yf);

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600,600);
    glutCreateWindow("Triangle Rotation about Fixed Point");

    init();
    glutDisplayFunc(display);
```

```
    glutMainLoop();
}
```

OUTPUT:



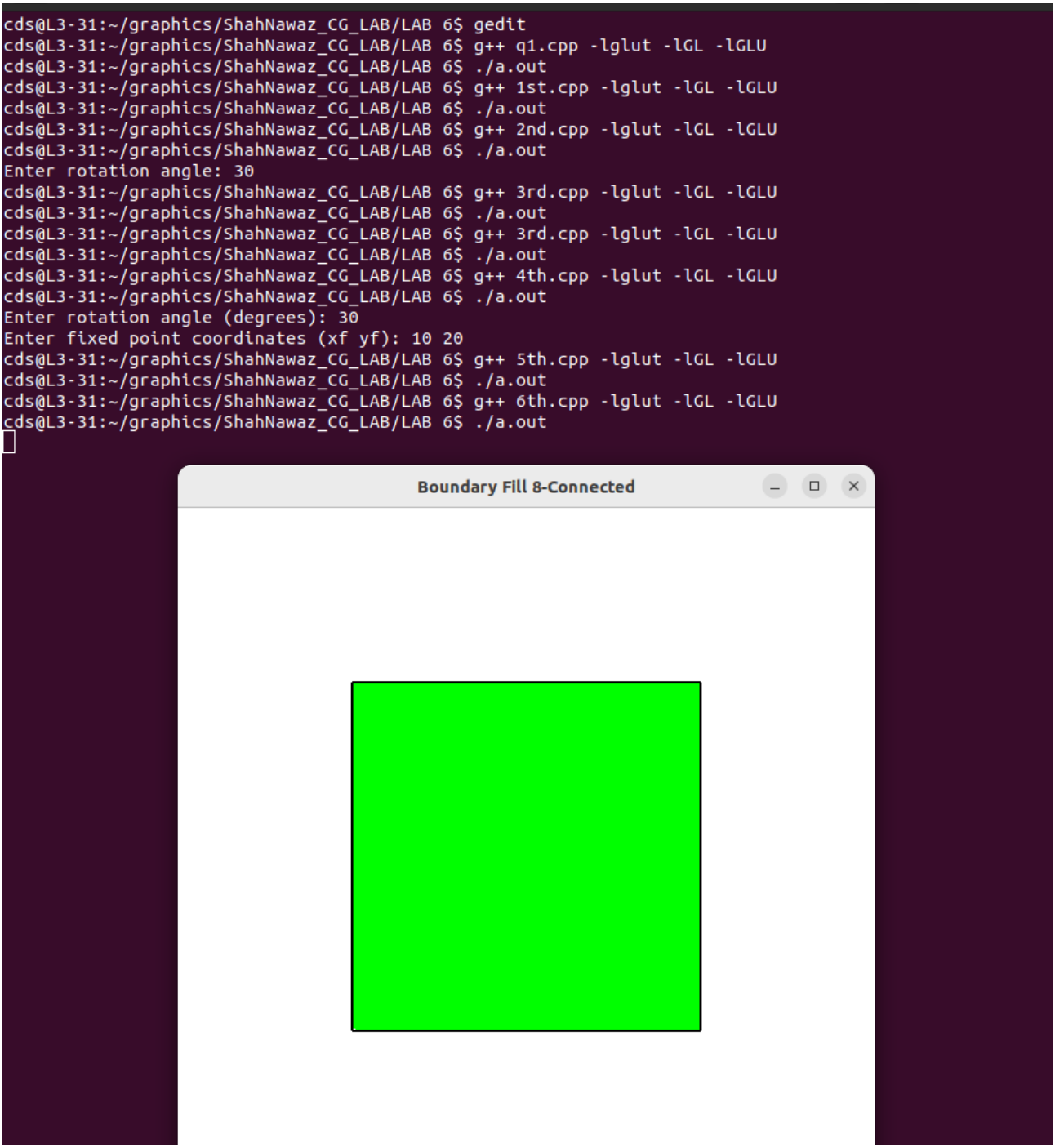
Q5. Write a program to implement **Boundary Fill Algorithm using recursion (8-connected)**

Ans.5
CODE:

```
#include <GL/glut.h>

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glColor3f(0.23f, 0.89f, 0.64f);
   glBegin(GL_LINE_LOOP);
      glVertex2f(0.0f, 0.0f);
      glVertex2f(6.0f, 0.0f);
      glVertex2f(6.0f, 9.0f);
      glVertex2f(0.0f, 9.0f);
   glEnd();


   glColor3f(0.76f, 0.40f, 0.33f);
   glBegin(GL_LINE_LOOP);
      glVertex2f(0.0f, 0.0f);
      glVertex2f(2.0f, 0.0f);
      glVertex2f(1.0f, 1.0f);
   glEnd();

   glColor3f(1.0f, 1.0f, 0.0f);
   glBegin(GL_LINE_LOOP);
      glVertex2f(5.0f, 0.0f);
      glVertex2f(11.0f, 0.0f);
      glVertex2f(11.0f, 9.0f);
      glVertex2f(5.0f, 9.0f);
   glEnd();


   glColor3f(1.0f, 0.0f, 0.0f);
   glBegin(GL_LINE_LOOP);
      glVertex2f(5.0f, 9.0f);
      glVertex2f(11.0f, 9.0f);
      glVertex2f(8.0f, 12.0f);
   glEnd();

   glFlush();
}

void init() {
   glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(-1.0, 15.0, -1.0, 12.0);
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Rectangle and Triangle Transformation");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Q6.
Write a program to:

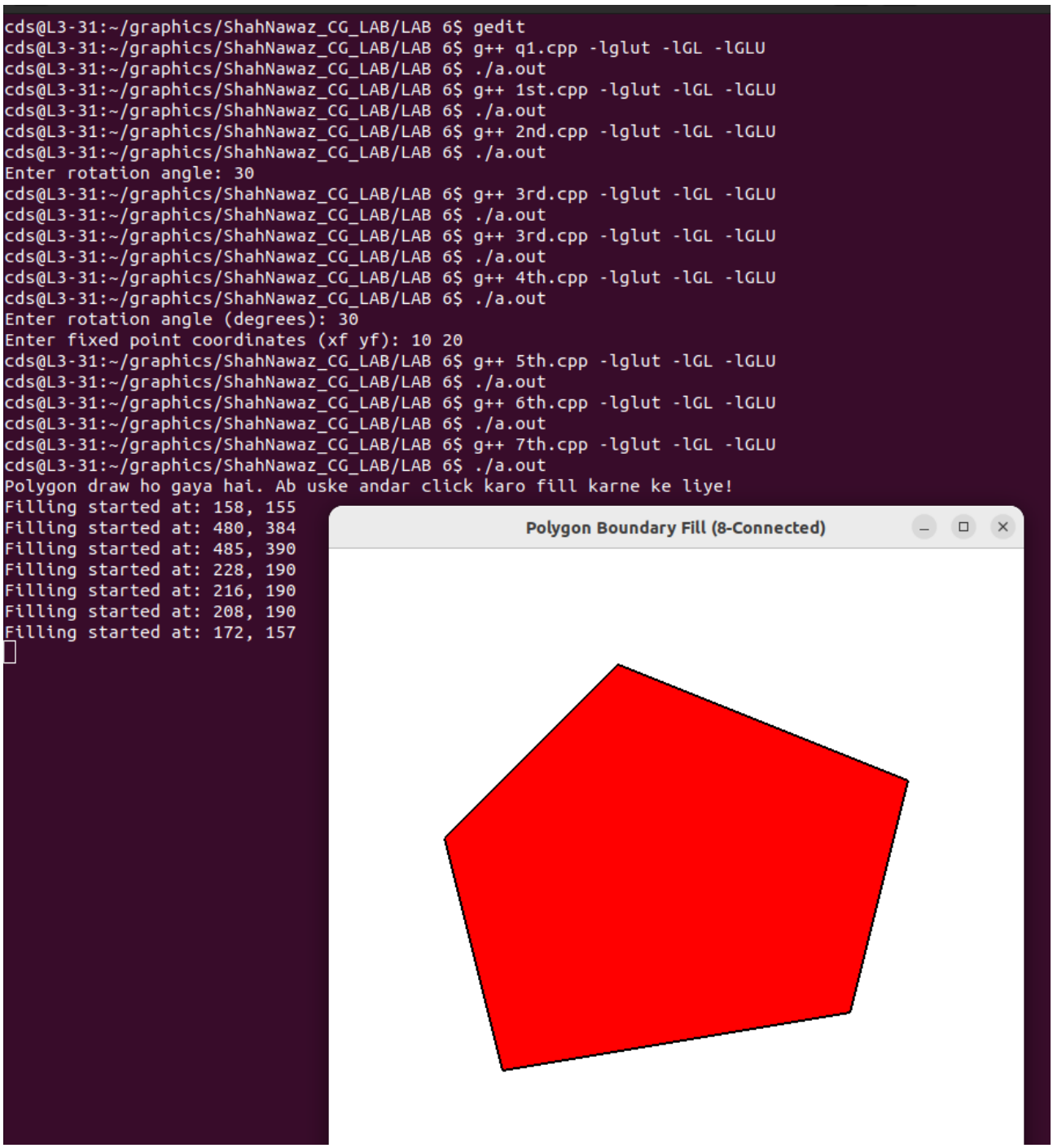
- Draw a polygon
- Fill it using Boundary Fill
- **Allow user to click seed point using mouse**

Ans.6
CODE:

```
#include <GL/glut.h>
#include <iostream>

float boundaryColor[3] = {0.0, 0.0, 0.0};
float fillColor[3] = {0.0, 1.0, 0.0};

void boundaryFill8(int x, int y, float* fillC, float* boundC) {
    float interiorColor[3];
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, interiorColor);

    if ((interiorColor[0] != boundC[0] || interiorColor[1] != boundC[1] || interiorColor[2] !=
boundC[2]) &&
        (interiorColor[0] != fillC[0] || interiorColor[1] != fillC[1] || interiorColor[2] != fillC[2])) {

        glColor3fv(fillC);
        glBegin(GL_POINTS);
            glVertex2i(x, y);
        glEnd();
        glFlush();


        boundaryFill8(x + 1, y, fillC, boundC);
        boundaryFill8(x - 1, y, fillC, boundC);
        boundaryFill8(x, y + 1, fillC, boundC);
        boundaryFill8(x, y - 1, fillC, boundC);
        boundaryFill8(x + 1, y + 1, fillC, boundC);
        boundaryFill8(x - 1, y + 1, fillC, boundC);
        boundaryFill8(x + 1, y - 1, fillC, boundC);
        boundaryFill8(x - 1, y - 1, fillC, boundC);
    }
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
            int xi = x;
        int yi = 600 - y;
        boundaryFill8(xi, yi, fillColor, boundaryColor);
    }
}
```

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);


    glColor3fv(boundaryColor);
    glLineWidth(2.0);
    glBegin(GL_LINE_LOOP);
        glVertex2i(150, 150);
        glVertex2i(450, 150);
        glVertex2i(450, 450);
        glVertex2i(150, 450);
    glEnd();

    glFlush();
}

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 600, 0, 600);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Boundary Fill 8-Connected");
    init();
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutMainLoop();
    return 0;
}
```

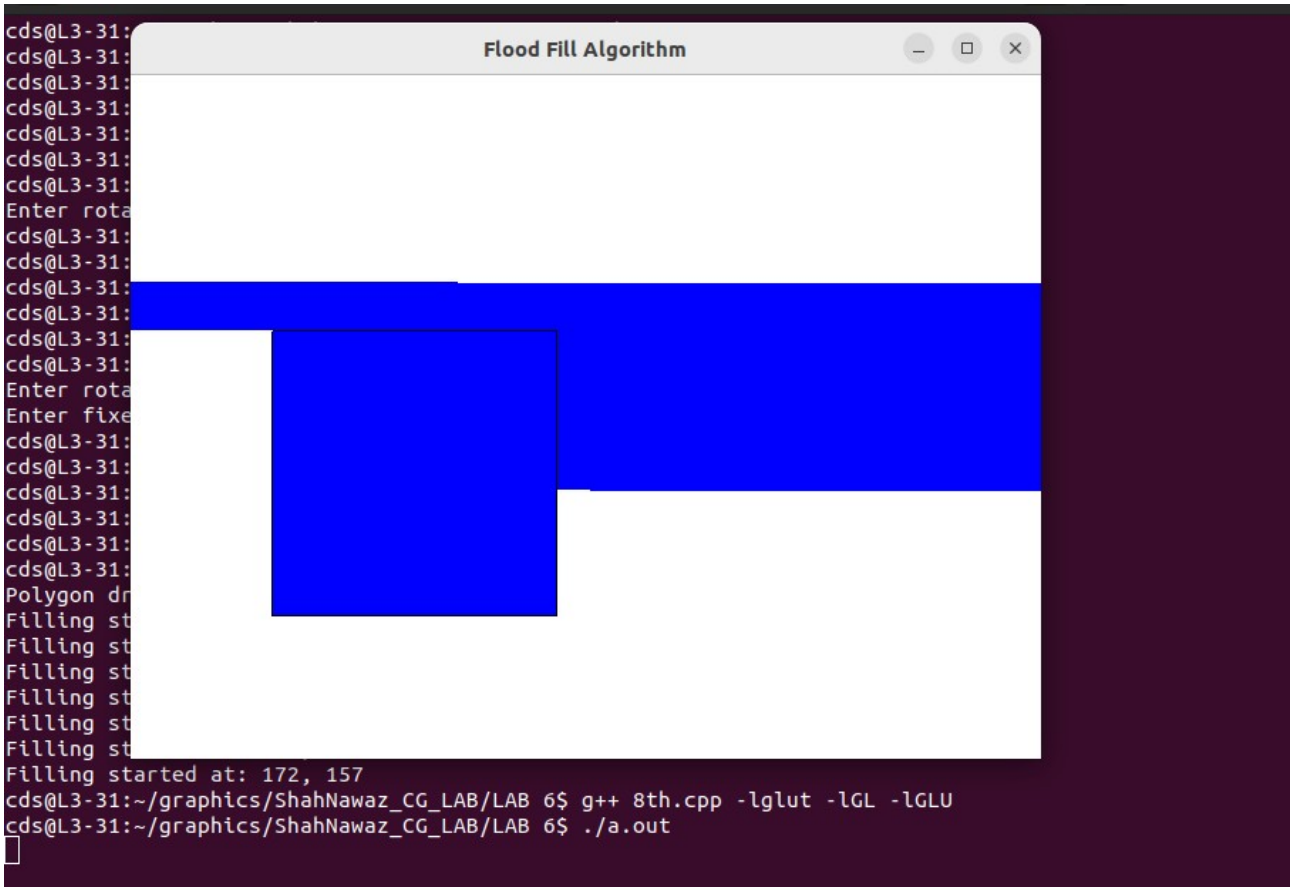
OUTPUT:



Q7. Write a program for **Flood Fill Algorithm**
Ans.7
CODE:

```
#include <GL/glut.h>
#include <iostream>
```

```
float oldColor[3] = {1.0, 1.0, 1.0};
float newColor[3] = {0.0, 0.0, 1.0};

void floodFill4(int x, int y, float* oldC, float* newC) {
    float currentColor[3];
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, currentColor);


    if (currentColor[0] == oldC[0] && currentColor[1] == oldC[1] && currentColor[2] == oldC[2])
{

        glColor3fv(newC);
        glBegin(GL_POINTS);
            glVertex2i(x, y);
        glEnd();
        glFlush();

        floodFill4(x + 1, y, oldC, newC);
        floodFill4(x - 1, y, oldC, newC);
        floodFill4(x, y + 1, oldC, newC);
        floodFill4(x, y - 1, oldC, newC);
    }
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        int xi = x;
        int yi = 480 - y;
        floodFill4(xi, yi, oldColor, newColor);
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0, 0.0, 0.0);
    glBegin(GL_LINE_LOOP);
        glVertex2i(100, 100);
        glVertex2i(300, 100);
        glVertex2i(300, 300);
        glVertex2i(100, 300);
    glEnd();

    glFlush();
}

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 640, 0, 480);
```

```
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutCreateWindow("Flood Fill Algorithm");
    init();
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutMainLoop();
    return 0;
}
```

OUTPUT: